CAIO ACADEMY

OPÉRATIONS

Architecture Infrastructure CAIO Academy

VPS dédié, Convex multi-projets, dashboards Next.js par C-Level, allowlist email, orchestration multi-rôles — l'architecture technique complète

AUTEUR CAIO Academy

DATE 4 mai 2026

ÉDITION Édition 2026

Sommaire

Infrastructure & architecture technique – CAIO Academy

Vue d’ensemble

Le VPS dedie

Authentification IA

Backend Convex multi-projets

Dashboards Next.js

Email allowlist

Orchestration multi-roles

Integrations & automatisations

Stack resumee

Schema de deploiement

Pourquoi cette architecture

© 2026 CAIO Academy

Infrastructure & architecture technique – CAIO Academy

> *Source de verite sur la facon dont l'equipe CAIO Academy developpe, deploie et opere ses dashboards C-Level. L'architecture est volontairement centralisee : un VPS dedie, un compte Convex parent, N projets Convex (un par C-Level), N dashboards Next.js sur Vercel, le tout filtre par allowlist email. Tout le travail de developpement et d'orchestration se fait sur cette machine, jamais en local.*

Vue d'ensemble

L'architecture repose sur cinq couches : acces, calcul, IA, backend, frontend. Chaque couche a une seule responsabilite et un seul point d'entree, ce qui rend le systeme inspectable, reproductible et auditable.

© 2026 CAIO Academy

```
+-------------------------------------------------------------+
|                        Membres de l'equipe                  |
|              (developpeurs + orchestrateurs)                |
+----------------------+-------------------------------------+
                       | SSH (port 42820, key-only)
                       v
+-------------------------------------------------------------+
|                        VPS dedie                            |
|   utilisateur unique : hacker      shell : zsh              |
|   tmux multi-sessions  |  git  |  Node 20  |  Bun          |
|   Convex CLI  |  Vercel CLI  |  Claude Code CLI             |
+----------------------+-------------------------------------+
                       |
                       v
+-------------------------------------------------------------+
|              Pool de credentials Claude Code                |
|   N comptes Claude Code Max (forfait illimite) +            |
|   ANTHROPIC_API_KEY (fallback / outils non-Max)             |
|   commandes : /switch, /switch-add                          |
+----------------------+-------------------------------------+
                       |
                       v
+-------------------------------------------------------------+
|              Compte Convex parent (1 organisation)          |
|  +---------+---------+---------+---------+---------+        |
|  |  CTO    |  CMO    |  CPO    |  CFO    |  CHRO   |  ...   |
|  | project | project | project | project | project |        |
|  +----+----+----+----+----+----+----+----+----+----+        |
+-------|---------|---------|---------|---------|-------------+
        |         |         |         |         |
        v         v         v         v         v
   +--------+ +--------+ +--------+ +--------+ +--------+
   |  CTO   | |  CMO   | |  CPO   | |  CFO   | | CHRO   |
   |  dash  | |  dash  | |  dash  | |  dash  | |  dash  |
   |Next.js | |Next.js | |Next.js | |Next.js | |Next.js |
   +---+----+ +---+----+ +---+----+ +---+----+ +---+----+
       |          |          |          |          |
       +----------+----+-----+----------+----------+
                       v
              +----------------+
              | Vercel (1 deploy/dashboard)        |
              | Domaines : <role>.caio-academy.x   |
              +-------+----------------------------+
                      |
                      v
              +-----------------------------+
              | Email allowlist gate        |
```


© 2026 CAIO Academy

```
                | (1 liste par C-Level dash)  |
                +-----------------------------+
```

Le principe directeur : un C-Level = un projet Convex = un dashboard Next.js = une allowlist. Cette correspondance 1-1-1-1 evite tout couplage croise et simplifie la gestion des permissions, des quotas et des incidents.

Le VPS dedie

Le coeur du systeme est un VPS Ubuntu chez un fournisseur europeen, accessible uniquement par SSH (port non-standard, authentification par cle). Tout le code, toute l'orchestration, toutes les sessions tmux vivent ici. Aucun developpeur ne pousse depuis sa machine personnelle.

Pourquoi centraliser

- **Reproductibilite** : un seul environnement, un seul Node, un seul Bun, un seul Vercel CLI configure. Pas de "ca marche chez moi".
- **Securite** : les cles Convex et Vercel ne quittent jamais le VPS. Les laptops perdus n'exposent rien.
- **Continuite** : tmux conserve les sessions actives apres deconnexion. Un build de 40 minutes continue meme si l'operateur ferme son ordinateur.
- **Observabilite** : tous les logs, tous les processus, tous les jobs cron sont au meme endroit, lisibles par toute l'equipe.

Acces equipe

Chaque membre de l'equipe a sa propre cle SSH ajoutee au compte `hacker`. La rotation de cles est manuelle : suppression dans `~/.ssh/authorized_keys` quand un membre quitte le projet.

```
# Connexion type
ssh -p 42820 hacker@<vps-ip>

# Rejoindre une session tmux existante (ex. dashboard CTO en cours)
tmux attach -t CAIO-CTO-dev

# Lister les sessions
tmux ls
```

Conventions tmux

Les sessions sont nommees selon le pattern `CAIO-<role>-<type>` :

- `CAIO-CTO-dev` : dev quotidien du dashboard CTO

© 2026 CAIO Academy

- `CAIO-CMO-fix-auth` : worker dispatche pour corriger l'auth du dashboard CMO
- `CAIO-oracle-1` : Oracle d'orchestration (voir section orchestration)

Le nommage permet a tout operateur de retrouver instantanement le contexte d'une session sans demander.

Authentification IA

CAIO Academy utilise Claude Code intensivement, pour le developpement comme pour l'orchestration. Deux modes d'authentification coexistent :

Comptes Claude Code Max (mode principal)

Plusieurs comptes Claude Code Max sont mis en commun dans un pool. Chaque compte a un quota de tokens illimites dans la limite du fair-use, ce qui evite la facturation API a la token. Pour des sessions de plusieurs heures par jour, l'economie est de l'ordre de 90 a 95% par rapport a une consommation API equivalente.

Le pool est gere par deux commandes simples :

```
# Ajouter un nouveau compte au pool
/switch-add

# Basculer vers le compte suivant (rotation)
/switch claude-account-2
```

La rotation manuelle permet d'eviter le rate-limiting d'un compte unique pendant des charges lourdes (audits Quality Arsenal, generations massives de contenu). Chaque compte est identifie par un nom court, isole dans un repertoire de credentials separe.

Cle API Anthropic (fallback)

Pour les outils qui ne supportent pas l'authentification Claude Code (scripts batch, integrations CI/CD, agents headless), une cle `ANTHROPIC_API_KEY` est disponible dans `~/.zshrc.private`. Cette cle facture a l'usage et est reservee aux automatisations qui ne peuvent pas passer par le pool Max.

© 2026 CAIO Academy

Quand utiliser quoi

Cas d'usage	Auth	Raison
Dev interactif quotidien	Claude Code Max	Forfait, pas de surprise de facturation
Audit Quality Arsenal (16 audits)	Claude Code Max	Volume eleve, forfait absorbe
Cron Convex generant un rapport	Cle API	Headless, pas de session interactive
Webhook reactif	Cle API	Pas de TTY, batch automatise

Backend Convex multi-projets

Convex est la source de verite backend. La particularite est qu'on n'utilise pas un seul projet Convex pour tout : on a **un compte parent** (organisation Convex) qui contient **N projets**, un par C-Level.

Pourquoi un projet par C-Level

- **Isolation des donnees** : les donnees du dashboard CTO (incidents, deploys, repos) n'ont aucune raison d'etre lisibles depuis le dashboard CMO (campagnes, leads, GA4). La separation au niveau projet rend l'isolation native.
- **Quotas independants** : chaque projet a son propre quota de fonctions, de bandwidth, de stockage. Un pic de trafic CMO n'affecte pas le CTO.
- **Deploys independants** : un schema brise sur le projet CFO ne casse pas le projet CHRO. Chaque C-Level peut iterer a son rythme.
- **Cles API separees** : chaque dashboard Next.js a son propre `NEXT_PUBLIC_CONVEX_URL` et `CONVEX_DEPLOY_KEY`, ce qui simplifie la rotation et l'audit.

Organisation par role

C-Level	Projet Convex	Donnees principales
CTO	`caio-cto`	Repos, deploys, incidents, infra
CMO	`caio-cmo`	Campagnes, leads, GA4, ads spend
CPO	`caio-cpo`	Produits, releases, NPS, churn
CFO	`caio-cfo`	Budgets, MRR, runway, factures
CHRO	`caio-chro`	Effectifs, recrutements, paie

© 2026 CAIO Academy

L'ajout d'un C-Level futur (CISO, COO, etc.) suit la meme convention : nouveau projet Convex, nouveau dashboard Next.js, nouvelle allowlist.

Capacites Convex utilisees

Convex couvre tout ce qui est habituellement eclate sur 4 a 5 services :

- **Schema declaratif** dans `convex/schema.ts`, types generes automatiquement
- **Queries** : lectures reactives qui se re-executent quand les donnees changent
- **Mutations** : ecritures transactionnelles cote serveur
- **Actions** : appels externes (HTTP, IA, Stripe, etc.) sans bloquer les mutations
- **Cron** : jobs planifies (`convex/crons.ts`), executes par Convex sans serveur a maintenir
- **Webhooks** : `httpAction` qui recoivent des events Stripe, Clerk, GitHub
- **Real-time** : abonnements WebSocket natifs, pas de Pusher / Ably a configurer

```
// Exemple de schema (extrait fictif)
export default defineSchema({
  incidents: defineTable({
    severity: v.union(v.literal("p1"), v.literal("p2"), v.literal("p3")),
    title: v.string(),
    openedAt: v.number(),
    closedAt: v.optional(v.number()),
  }).index("by_severity", ["severity"]),
});
```

Dashboards Next.js

Chaque C-Level a un dashboard Next.js dedie, deploye sur Vercel. Le code source vit dans un sous-dossier du repo CAIO Academy (`apps/dashboard-cto`, `apps/dashboard-cmo`, etc.) et chaque app a son propre `next.config.ts`, son propre `package.json` de production et son propre projet Vercel.

Stack frontend

- **Next.js 16** (App Router, Cache Components, Turbopack)
- **React 19** + Server Components
- **Tailwind v4** + shadcn/ui pour le design system
- **Clerk** pour l'authentification (voir allowlist)
- **convex/react** pour la connexion temps reel au backend
- **Vercel Analytics** + **Vercel Blob** pour la metrique et le stockage actif

© 2026 CAIO Academy

Pourquoi un dashboard par role

Chaque C-Level a des besoins de visualisation specifiques. Le dashboard CFO affiche du financier (MRR, runway, cohort revenue) avec des graphes denses et un export CSV. Le dashboard CMO affiche des funnels, des heatmaps, des creatives. Mutualiser ces UIs dans une seule app conduit a une feature flag soup illisible et a des bundle sizes prohibitifs. Un dashboard par role = un bundle propre, un cycle de release independant, un design taillee pour les decisions de ce role.

Variables d'environnement type

```
# .env.local pour le dashboard CTO
NEXT_PUBLIC_CONVEX_URL=https://caio-cto.convex.cloud
CONVEX_DEPLOY_KEY=prod:...
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_live_...
CLERK_SECRET_KEY=sk_live_...
ALLOWLIST_TEAM=cto
```

Email allowlist

Chaque dashboard est ferme par defaut. Seuls les emails inscrits dans l'allowlist du C-Level correspondant peuvent se connecter. Toute tentative depuis un email non listee est refusee a la couche d'auth, avant tout chargement de donnees.

Mecanisme

Trois couches de defense, dans l'ordre :

1. **Clerk** : la liste d'emails autorises est configuree au niveau de l'application Clerk (mode "restricted access"). Aucun signup public, aucune inscription self-serve.
2. **Middleware Next.js** : verifie a chaque requete que l'email du JWT Clerk appartient a l'allowlist du dashboard, sinon redirige vers `/forbidden`.
3. **Convex** : chaque query / mutation re-verifie l'identite cote serveur via `ctx.auth.getUserIdentity()` et compare a la liste C-Level. Une cle API volee ne contourne pas la verification.

© 2026 CAIO Academy

Listes par role

Dashboard	Allowlist	Source
CTO	`team-cto.json`	maintenue par le CTO
CMO	`team-cmo.json`	maintenue par le CMO
CPO	`team-cpo.json`	maintenue par le CPO
CFO	`team-cfo.json`	maintenue par le CFO
CHRO	`team-chro.json`	maintenue par le CHRO

Les listes sont versionnees dans le repo. Toute modification passe par une PR, ce qui garantit l'auditabilite : on sait quand un email a ete ajoute ou retire, et par qui.

Pourquoi pas SSO

Un SSO complet (Okta, Google Workspace) est sur la roadmap An 2 quand l'equipe depassera 30 personnes. En Phase I, l'allowlist statique est suffisante : faible volume, rotation rare, audit simple par diff Git.

Orchestration multi-roles

Le developpement et l'operation reposent sur un pattern Oracle / workers herite de l'ecosysteme Agentik OS, adapte a CAIO Academy.

Roles

- **Oracle** : session tmux longue duree par projet (`CAIO-Academy-oracle-1`). Recoit les missions, classifie (SIMPLE / MEDIUM / COMPLEX / EPIC), planifie, dispatche.
- **Worker** : session tmux ephemere (`CAIO-Academy-worker-4-1`). Execute une mission bornee, ecrit `~/.aisb/state/worker-<session>.done.json` quand termine.
- **Patrol** : daemon qui lit toutes les `progress.json` des workers et notifie l'Oracle / l'utilisateur en cas de blocage.
- **Verifier** : worker special (Guardian) qui verifie le build, les screenshots et le perimetre de fichiers avant cloture.

© 2026 CAIO Academy

Conventions de session

Pattern	Role	Duree
`CAIO-Academy-oracle-N`	Oracle persistant	Jours / semaines
`CAIO-<role>-dev`	Dev interactif sur un dashboard	Heures
`CAIO-Academy-worker-N-M`	Worker dispatche	Minutes / heures
`CAIO-Academy-fix-<scope>`	Fix specifique	Minutes

Cycle de vie d'une mission

1. Un membre de l'equipe ouvre une session Oracle ou y rattache.
2. L'Oracle classifie la mission. Si COMPLEX, il invoque `/team` qui spawn N teammates en panes tmux.
3. Chaque worker execute sa portion, met a jour son `progress.json`, ecrit `done.json` a la fin.
4. Le Verifier valide le build, les artefacts, le perimetre.
5. L'Oracle ferme la session (graceful shutdown via `TeamDelete`) et reporte au membre de l'equipe.

Aucun worker n'edite directement la prod. Le push, le merge et le deploy sont des etapes explicites declenchees par l'Oracle apres la verification.

Integrations & automatisations

Tout ce qui est "evenementiel" ou "planifie" passe par Convex, pas par un service externe. Cela garde la pile cohesive et inspectable depuis un seul dashboard.

Cron Convex

Les jobs recurrents sont declares dans `convex/crons.ts` :

© 2026 CAIO Academy

```
import { cronJobs } from "convex/server";
import { internal } from "./_generated/api";

const crons = cronJobs();

crons.weekly(
  "metrics digest",
  { dayOfWeek: "friday", hourUTC: 15, minuteUTC: 0 },
  internal.reports.weeklyDigest,
);

export default crons;
```

Exemples concrets : digest hebdomadaire des KPIs envoye par email, recompute des cohortes de retention chaque nuit, refresh des stats LinkedIn / Beehiiv toutes les 6 heures.

Webhooks entrants

Les `httpAction` Convex servent d'endpoints HTTPS pour les webhooks Stripe, Clerk, GitHub, Vercel, ElevenLabs. Chaque endpoint verifie la signature de l'expediteur avant ecriture, et journalise l'event brut dans une table `webhook_events` pour replay.

Real-time

Les dashboards consomment des queries Convex via `useQuery`. Lorsqu'une mutation modifie une ligne, toutes les UIs ouvertes se re-rendent en temps reel sans WebSocket custom. C'est la fonctionnalite qui rend Convex strategique pour des dashboards executifs : le CTO voit un incident apparaitre des qu'il est cree, sans rafraichir.

Integrations IA

Les agents Claude / OpenAI / ElevenLabs sont appeles depuis des `action` Convex. Les actions ne sont pas transactionnelles, donc elles peuvent appeler du HTTP externe sans verrouiller la base. Une mutation appelle l'action, l'action recupere la reponse IA, puis appelle une seconde mutation pour persister le resultat.

© 2026 CAIO Academy

Stack resumee

Couche	Outil	Role	Notes
Compute	VPS dedie	Dev + orchestration	SSH only, port custom
Sessions	tmux	Persistance shells	Conventions `CAIO-<role>-<type>`
IA	Claude Code Max + API	Auth IA mutualisee	Pool de comptes, `/switch`
Backend	Convex (1 parent, N projets)	DB + functions + cron + webhooks	1 projet par C-Level
Auth	Clerk + allowlist	Filtrage email strict	Listes versionnees Git
Frontend	Next.js 16 (App Router)	Dashboards par C-Level	Tailwind + shadcn/ui
Hosting	Vercel	Deploy edge	1 projet Vercel par dashboard
Observabilite	Vercel Analytics + logs Convex	Metriques + traces	Centralisation native

---

Schema de deploiement

Le flux est unidirectionnel : developpement local sur VPS, push GitHub, deploy automatique Vercel + Convex.

© 2026 CAIO Academy

Etape	Acteur	Action	Cible
1	Developpeur (worker)	Edits dans `apps/dashboard-<role>/`	Repo local sur VPS
2	Verifier	`npm run build` et tests	VPS
3	Oracle	`git add` + `git commit` cible	Repo local
4	Oracle	`git push`	GitHub
5	Vercel (auto)	Build + deploy Next.js	`https://<role>.caio-academy.x`
6	Convex CLI	`npx convex deploy --prod`	Convex prod du projet concerne
7	Oracle	Smoke check (curl 200, screenshot Playwright)	Prod
8	Oracle	Notifie l'equipe (Telegram / email)	–

L'etape 6 est decouplee de l'etape 5 : un changement frontend pur ne requiert pas de redeploy Convex, et inversement. Cela reduit le rayon d'impact de chaque release.

Regle de freeze

Si un deploy echoue (build rouge, healthcheck KO), le projet est "frozen" : aucune autre release ne peut partir avant que la cause soit identifiee. Pas de rollback automatique : on prefere un freeze explicite a un rollback aveugle qui pourrait masquer la racine du probleme.

Pourquoi cette architecture

Trois principes guident l'ensemble :

1. **Centralisation operationnelle** : un seul endroit pour developper et orchestrer, donc un seul endroit a securiser et a monitorer.
2. **Isolation par role** : un projet Convex et un dashboard Next.js par C-Level evitent le couplage croise. Chaque equipe avance a son rythme sans casser les autres.
3. **Auditabilite** : allowlist Git, sessions tmux nommees, jobs cron declaratifs, webhooks logges. Tout est tracable, replayable, reversible.

Le compromis explicite : on accepte la complexite de gerer N projets Convex et N dashboards plutot qu'un monolithe, parce que la separation paye sur la securite, la velocite par equipe et la lisibilite

© 2026 CAIO Academy

operationnelle. Cette architecture tient jusqu’a une dizaine de C-Levels et une centaine de membres d’equipe avant de necessiter SSO et federation multi-VPS.

© 2026 CAIO Academy